

Lecture 3: Supervised Learning on Images
Convolutional Neural Networks and Regularisation
Class Activation Maps, Adversarial Examples, Limits

Deep Learning for Scientific Discovery

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków



Doctoral School
of Exact and
Natural Sciences
Jagiellonian University

Recap: MNIST, the Deep Learning "Hello World"

1 output, MSE

$\mathcal{L}_{\text{MSE}} = (\bar{y} - y)^2$

$y \in \{0, \ldots, 9\}$: true digit, $\bar{y}$: network output

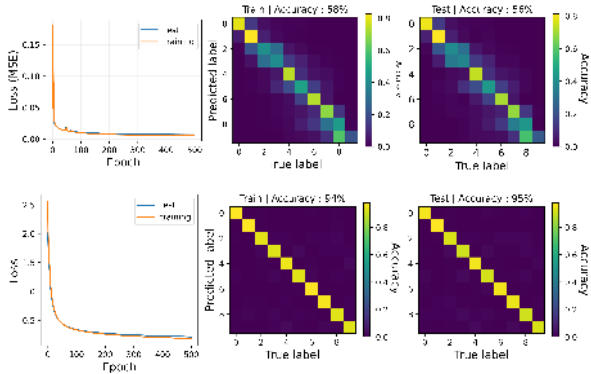
test accuracy 56%

10 outputs, cross-entropy

$\mathcal{L}_{\text{CE}} = -\log q(\text{true digit})$

$q(k)$: softmax output for digit k

test accuracy 95%



Losses averaged over the training set; same hidden layer (21 sigmoid nodes). Confusion matrix: row = predicted, column = true digit. One output orders the digits: MSE errors lie next to the diagonal (3 for 2, 9 for 8).

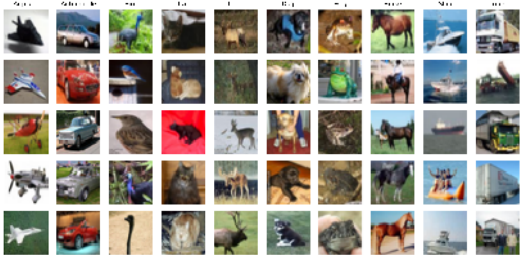
the formulation of the problem matters as much as the network

1

Hello World #2: CIFAR-10 and Convolutional Networks

JAGIELLONIAN UNIVERSITY
IN KRAKOW

CIFAR-10 Dataset



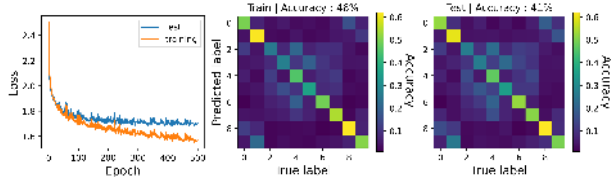
- 10 classes: aeroplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
- 50 000 training, 10 000 test images
- 32×32 pixels in **3 colour channels** (R, G, B): each image is a $32 \times 32 \times 3$ tensor
- MNIST was $28 \times 28 \times 1$

A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images* (2009), cs.toronto.edu/~kriz/cifar.html

CIFAR-10: The Same Naive Network + Cross-Entropy

Exactly the MNIST recipe:

- **flatten** the image:
  $32 \cdot 32 \cdot 3 = 3072$ inputs
- one hidden layer, 21 sigmoid nodes
- 10 outputs, softmax, cross-entropy



network and curves: Ch. 2 of Heightman & Płodzień, Springer (to appear, 2027), arXiv:2508.10666

$3072 \cdot 21 + 21 + 21 \cdot 10 + 10 = 64\,753$ parameters

- **Poor performance:** about 46% training, 41% test accuracy — the model *underfits*

What do we lack in our approach?

Flattening Loses Spatial Structure

Neighbouring pixels are correlated: an edge, a texture, an eye are *local* patterns of a few nearby pixels.

After flattening, the dense layer treats pixel #1 and pixel #2 as unrelated as pixel #1 and pixel #3000.

Shuffle all pixels with one fixed permutation: a dense network learns equally well — it never used the 2D layout.

What an image model should know

- **locality:** look at small neighbourhoods
- **weight sharing:** the same edge detector is useful everywhere in the image
- **translation:** a cat in the corner is still a cat

$\Rightarrow$ **capture spatial correlations** with *convolutions*.

Convolution: the Continuous Definition (1D)

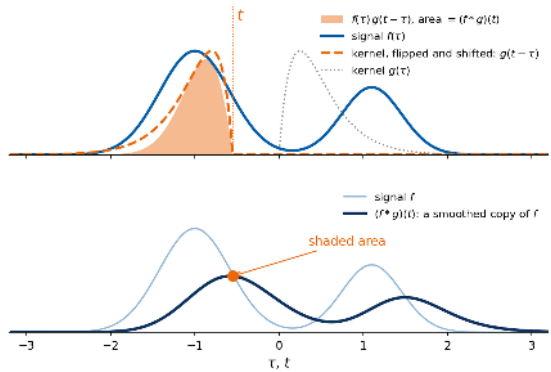
Two functions $f(\tau)$ (the signal) and $g(\tau)$ (the kernel):

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

For each output point t:

- **flip** the kernel, $g(-\tau)$
- **shift** it by t, $g(t-\tau)$
- **multiply** with $f(\tau)$
- **integrate**: the shaded area is $(f * g)(t)$

Moving t traces the curve $f * g$, here a **smoothed** copy of f.



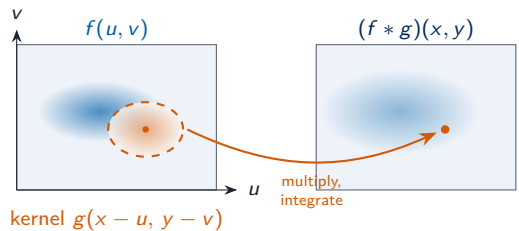
Convolution: the Continuous Definition (2D)

For functions of two variables, e.g. an image $f(x, y)$ and a kernel $g(x, y)$:

$$(f * g)(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(u, v)\, g(x - u,\, y - v)\, du\, dv$$

the same operations as in 1D: flip g in both directions, shift it to (x, y), multiply with f, integrate over the plane; one number for each point (x, y)

g a 2D Gaussian $\Rightarrow$ $f * g$ is f **blurred**: every point replaced by a weighted average of its neighbourhood



the kernel, centred at (x, y), weights the image around that point; the integral gives the output there

Convolution Theorem

In Fourier space a convolution is a product

$$\mathcal{F}[f](k) = \int f(t)\, e^{-ikt}\, dt, \qquad \mathcal{F}[f * g](k) = \mathcal{F}[f](k) \cdot \mathcal{F}[g](k)$$

$$f * g = \mathcal{F}^{-1}\big[\mathcal{F}[f] \cdot \mathcal{F}[g]\big]$$

in 2D the same, with $e^{-i(k_x x + k_y y)}$

Further properties of the convolution:

- linear
- commutative, $f * g = g * f$
- shift-equivariant, $f(\cdot - a) * g = (f * g)(\cdot - a)$

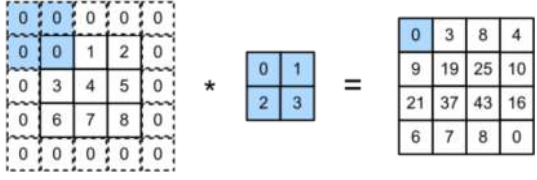
Discrete Convolution in 2D

Image $x(i,j)$, kernel $G(m,n)$ of size $k \times k$:

$$\text{convolution:} \quad (G * x)(i,j) = \sum_m \sum_n G(m,n)\, x(i-m,\, j-n)$$

$$\text{cross-correlation (what CNN layers compute):} \quad S(i,j) = \sum_m \sum_n G(m,n)\, x(i+m,\, j+n)$$

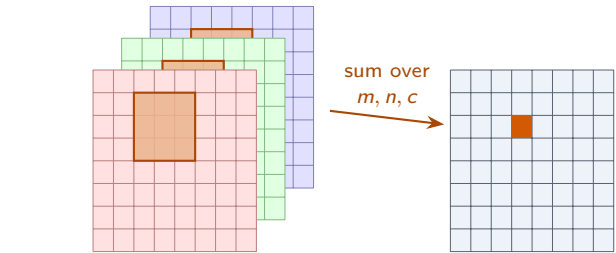
The two differ only by flipping the kernel. The kernel is *learned*, so the flip does not matter: deep-learning libraries compute the cross-correlation and still call it "convolution".



a 2×2 kernel slides over a zero-padded 3×3 input; each output is the sum of element-wise products, e.g. $0 \cdot 0 + 0 \cdot 1 + 0 \cdot 2 + 1 \cdot 3 = 3$

Images with Three Channels

A colour image has three numbers per pixel, red, green and blue: $x(i, j, c)$ with $c \in \{R, G, B\}$. A CIFAR-10 image is $32 \times 32 \times 3$.



input $32 \times 32 \times 3$: red, green, blue layers; kernel $3 \times 3 \times 3$ (orange)

one feature map (one channel)

$$S(i, j \mid c) = \sum_{m,n} G(m, n, c) \times x(i + m, j + n, c)$$

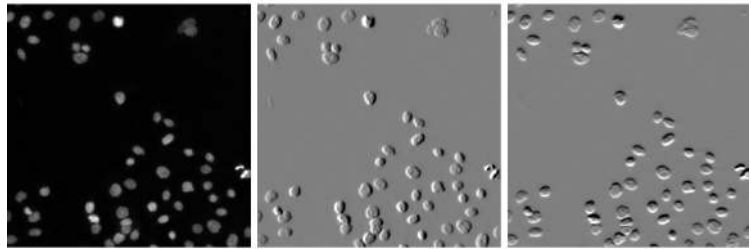
$$S(i, j) = \sum_{c} S(i, j \mid c) + b$$

each channel is convolved with its own slice of the kernel, and the three results are added: one kernel gives *one* feature map; C_{out} kernels give C_{out} feature maps, which are the channels of the next layer's input

Kernels Detect Features: Sobel Filters

$$\vec{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

$$\vec{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$



human cell nuclei, fluorescence microscope · vertical edges ($\vec{G}_x$) · horizontal edges ($\vec{G}_y$)

The Sobel kernels approximate the intensity gradient: they light up where the image changes sharply, at **edges**. Sharpening, blurring and the identity are 3×3 kernels too.

Idea of a CNN: do not design the kernels, learn them.

image: D. Root, CC0, from J. Moffat et al., *A Lentiviral RNAi Library for Human and Mouse Genes Applied to an Arrayed Viral High-Content Screen*, Cell **124**, 1283 (2006), doi:10.1016/j.cell.2006.01.040

Kernels Are Learnable Parameters

-1	0	$+1$
-2	0	$+2$
-1	0	$+1$

Sobel $\vec{G}_x$:
fixed, designed by a person

G_{11}	G_{12}	G_{13}
G_{21}	G_{22}	G_{23}
G_{31}	G_{32}	G_{33}

CNN kernel:
9 trainable weights

- the entries $G(m, n, c)$ and the bias b are the **weights** of a convolutional layer, as W and b are for a dense layer
- they start random and are trained by gradient descent on the same loss:

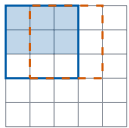
$$G(m, n, c) \leftarrow G(m, n, c) - \eta \frac{\partial \mathcal{L}}{\partial G(m, n, c)}$$

- backpropagation gives the gradient; one kernel is shared by all positions, so its gradient sums over all of them

After training, first-layer kernels often look like edge detectors.

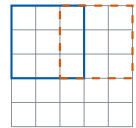
Kernel Size, Stride, Padding

stride 1



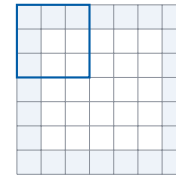
5×5 input, 3×3 kernel $\rightarrow$ 3×3 output

stride 2



the kernel jumps 2 pixels $\rightarrow$ 2×2 output

padding 1



zeros around the border $\rightarrow$ 5×5 output ("same")

Output size along one dimension

$$O = \left\lfloor \frac{W - k + 2P}{s} \right\rfloor + 1$$

W : input size, k : kernel size
P : padding, s : stride

CIFAR-10: $W = 32$, $k = 3$, $P = 1$, $s = 1 \Rightarrow O = 32$; with $s = 2$: $O = 16$

Pooling: Max, Average, Global Average

Window 2×2, stride 2: each 2×2 block of the feature map becomes *one* number.

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 4 & 2 & 1 & 3 \\ 9 & 8 & 6 & 5 \end{bmatrix} \xrightarrow{\text{max pooling}} \begin{bmatrix} 6 & 8 \\ 9 & 6 \end{bmatrix}$$

$$\xrightarrow{\text{average pooling}} \begin{bmatrix} 3.75 & 5.25 \\ 5.75 & 3.75 \end{bmatrix}$$

$$\xrightarrow{\text{global average pooling}} 4.625$$

top-left block: $\max(1,3,5,6) = 6$, $\quad (1+3+5+6)/4 = 3.75$; $\quad$ global: $74/16 = 4.625$

- **max pooling**: the strongest response; used between conv layers
- **global average pooling**: one number per feature map; used at the end of a network
- no trainable parameters

Pooling for Downsampling

A 2×2 pooling window with stride 2:

- 4 times fewer numbers to store and to process in every later layer
- the next 3×3 kernel covers a 6×6 region of the map before pooling
- small shifts of a feature do not change the output

The price

exact positions are lost

CNNs Learn a Hierarchy of Features

- a CNN is a neural network that **learns the kernels** for each channel
- layers of kernels capture a **hierarchy of features**: edges $\rightarrow$ parts $\rightarrow$ objects



first layer | middle layer | deep layer

features learned from face images by a convolutional deep belief network: H. Lee, R. Grosse, R. Ranganath, A. Y. Ng, *Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations*, ICML 2009, doi:10.1145/1553374.1553453

Second Attempt at CIFAR-10 Classification

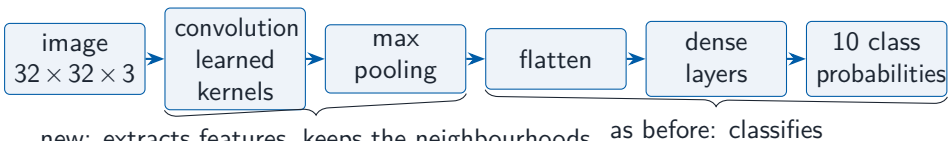
First attempt: dense network

- image flattened into 3072 numbers
- pixel neighbourhoods are lost
- 46% training, 41% test accuracy

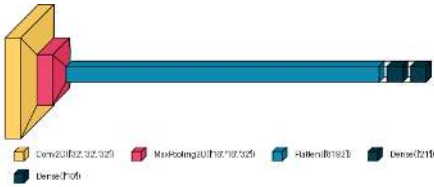
Second attempt: convolutional network

- image kept as $32 \times 32 \times 3$
- *learned* kernels on small neighbourhoods
- pooling shrinks the feature maps

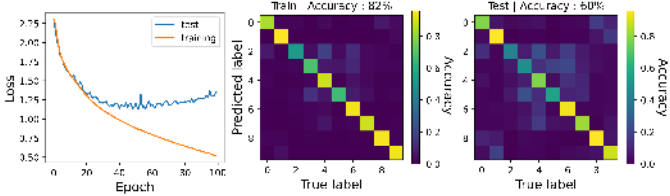
Same data, same loss (categorical cross-entropy), same training loop: **only the model changes.**



A Small CNN on CIFAR-10



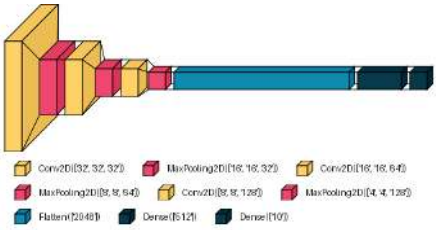
- Conv2D: 32 kernels 3×3, sigmoid, padding "same" $\rightarrow 32 \times 32 \times 32$
- MaxPooling $2 \times 2 \rightarrow 16 \times 16 \times 32$
- flatten $\rightarrow$ 8192, dense 21 (sigmoid), dense 10 (softmax)



network and curves: Ch. 2 of Heightman & Płodzień, Springer (to appear, 2027), arXiv:2508.10666

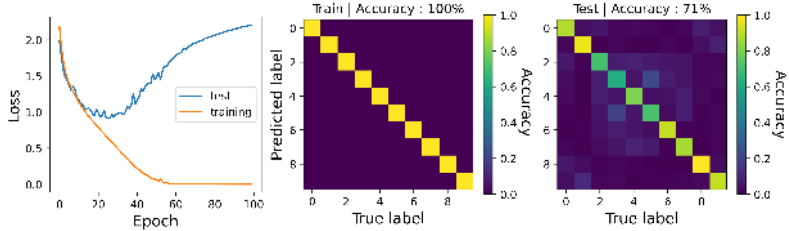
About 82% training, 60% test accuracy — better than 41%, but the test loss **rises after ∼40 epochs**: *overfitting*.

More CNN Layers: Still Overfitting



three blocks Conv2D (ReLU) + MaxPooling: $32 \times 32 \times 32 \to 16 \times 16 \times 64 \to 8 \times 8 \times 128$, then flatten, dense 512, dense 10

network and curves: Ch. 2 of Heightman & Płodzień, Springer (to appear, 2027), arXiv:2508.10666



100% training, 71% test accuracy: **more complexity does not cure overfitting**.

2

Regularisation: Curing Overfitting

JAGIELLONIAN UNIVERSITY
IN KRAKÓW

Regularisation: Three Common Techniques

Regularisation: any technique that constrains training or model capacity so that the network *generalises* to unseen data instead of memorising the training set.

L1 / L2 penalty	Dropout	Batch normalisation
a term in the loss that keeps the weights small	randomly switch off neurons during training	normalise the inputs of each hidden layer over a mini-batch

Also widely used: *early stopping* (stop when the validation loss starts to rise), *data augmentation* (shifted, flipped, cropped copies of the training images), *learning-rate scheduling*.

L1 / L2 Regularisation

L1 (Lasso)

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \lambda \sum_i |\theta_i|$$

drives some weights exactly to zero: *sparse* solutions

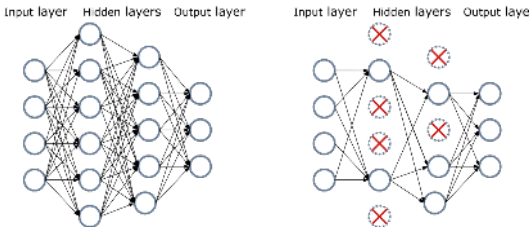
L2 (Ridge, "weight decay")

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \lambda \sum_i \theta_i^2$$

discourages large weights: *smoother* models

- θ_i: the weights of the network; λ: the regularisation strength, a hyperparameter
- a **soft constraint**: weights are encouraged, not forced, to stay small — λ plays the role of a Lagrange multiplier
- $\lambda = 0$: no regularisation; λ too large: the network underfits

Dropout



- at every training step each neuron of the layer is switched off with probability p (the *dropout rate*, typically 0.2–0.5)
- the network cannot rely on any single neuron: it learns **redundant representations**
- each step trains a different sub-network: dropout acts as an **ensemble** of sub-networks
- at test time all neurons are active

N. Srivastava et al., *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, JMLR **15**, 1929 (2014), jmlr.org/papers/v15/srivastava14a.html

Batch Normalisation

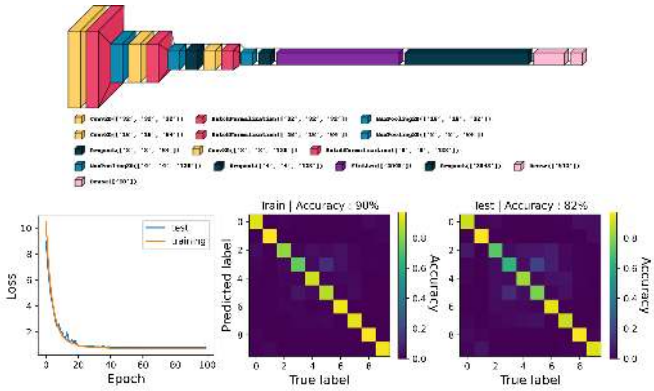
A BatchNorm layer normalises the input of a hidden layer over the mini-batch $\{\mathbf{x}_j\}_{j=1}^{N_B}$:

$$\mu_B = \frac{1}{N_B}\sum_{j=1}^{N_B}\mathbf{x}_j, \qquad \sigma_B^2 = \frac{1}{N_B}\sum_{j=1}^{N_B}(\mathbf{x}_j-\mu_B)^2 \qquad \mathbf{x}_j \leftarrow \frac{\mathbf{x}_j-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}, \qquad \mathbf{x}_j \leftarrow \gamma\,\mathbf{x}_j+\beta$$

- normalise: zero mean and unit variance within the mini-batch (ϵ: a small constant for numerical stability)
- then rescale and shift with **trainable** γ and β
- makes the network less dependent on weight initialisation
- reduces sensitivity to weight changes; helps against exploding and vanishing gradients

S. Ioffe, C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML 2015, arXiv:1502.03167

The Larger CNN with Regularisation



Same three Conv2D blocks, now with batch normalisation and dropout: 90% training, 82% test accuracy.

Before: 100% / 71%. **The training and test losses now stay together.**

network and curves: Ch. 2 of Heightman & Płodzień, Springer (to appear, 2027), arXiv:2508.10666

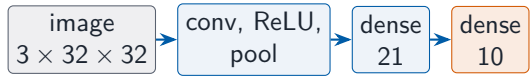
Lab 13a: Dense Network and Small CNN on CIFAR-10

dense network



layer	output	parameters
Flatten	3072	–
Linear	21	64 533
Linear	10	220
total		64 753

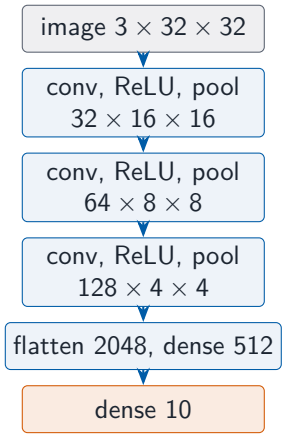
small CNN



layer	output	parameters
Conv2d	32 × 32 × 32	896
MaxPool2d	32 × 16 × 16	–
Flatten	8192	–
Linear	21	172 053
Linear	10	220
total		173 169

Sigmoid after the hidden dense layer, ReLU after the convolution (not listed, no parameters); all parameters trainable, none frozen. The lab trains its own versions of these networks; its results can differ from the book results above.

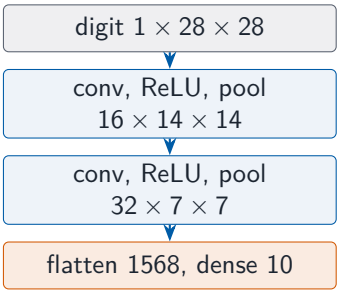
Lab 13a: Larger and Regularised CNN on CIFAR-10



layer	output	larger	regularised
Conv2d	$32 \times 32 \times 32$	896	896
BatchNorm2d	$32 \times 32 \times 32$		64
Conv2d	$64 \times 16 \times 16$	18 496	18 496
BatchNorm2d	$64 \times 16 \times 16$		128
Conv2d	$128 \times 8 \times 8$	73 856	73 856
BatchNorm2d	$128 \times 8 \times 8$		256
Linear	512	1 049 088	1 049 088
Linear	10	5 130	5 130
total		1 147 466	1 147 914

all trainable, none frozen; layers without parameters (ReLU, max pooling, flatten, dropout) not listed; batch normalisation and dropout only in the regularised CNN

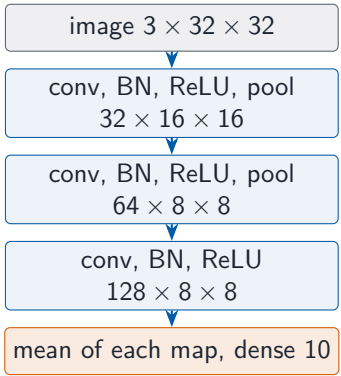
Lab 13a: CNN on MNIST



layer	output	parameters
Conv2d	$16 \times 28 \times 28$	160
MaxPool2d	$16 \times 14 \times 14$	–
Conv2d	$32 \times 14 \times 14$	4 640
MaxPool2d	$32 \times 7 \times 7$	–
Flatten	1568	–
Linear	10	15 690
total (all trainable)		20 490

ReLU after each convolution (no parameters, not listed)

Lab 13a: CNN with Global Average Pooling



layer	output	parameters
Conv2d	$32 \times 32 \times 32$	896
BatchNorm2d	$32 \times 32 \times 32$	64
Conv2d	$64 \times 16 \times 16$	18 496
BatchNorm2d	$64 \times 16 \times 16$	128
Conv2d	$128 \times 8 \times 8$	73 856
BatchNorm2d	$128 \times 8 \times 8$	256
Linear	10	1 290
total (all trainable)		94 986

ReLU and max pooling not listed (no parameters); the mean of each map is computed in `forward`, not by a layer

3

Class Activation Maps: Where the CNN Looks

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Class Activation Maps



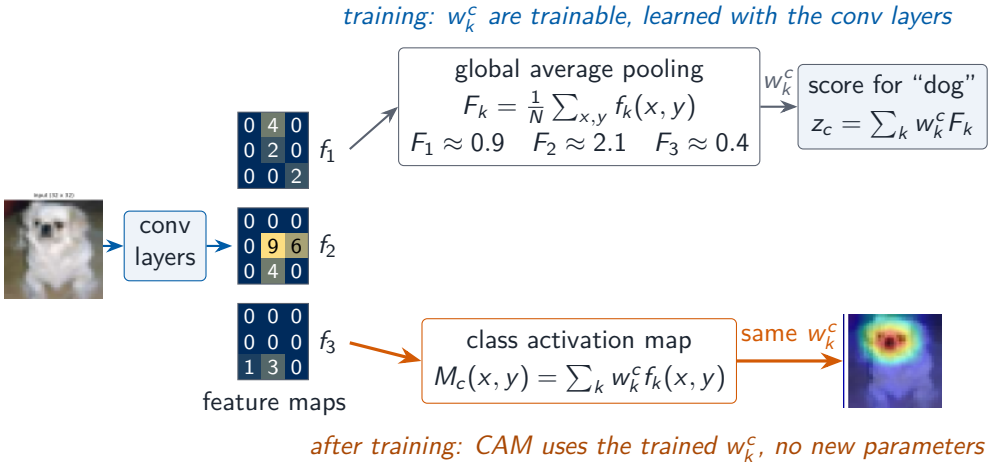
a CIFAR-10 test image; our CNN: "dog", 1.00

Which parts of the image were important for the prediction?

Can we get insight into what the CNN was looking at when it made its prediction?

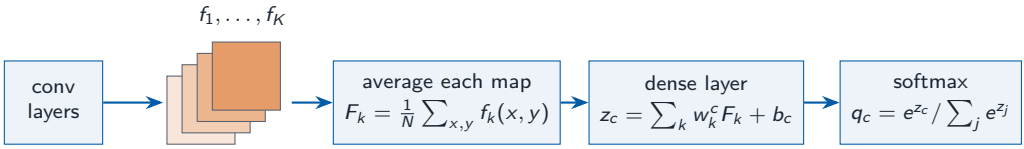
A **class activation map** (CAM) answers with a picture: a map over the image that shows *where* the evidence for a class is.

Feature Maps and the Class Activation Map



B. Zhou et al., *Learning Deep Features for Discriminative Localization*, CVPR 2016, arXiv:1512.04150

CAM: Decomposition of the Class Score



K feature maps, each $h \times w$, $N = hw$ pixels

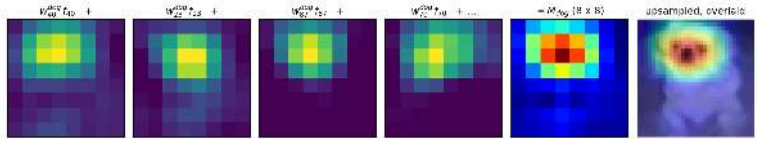
$$z_c = \sum_k w_k^c F_k + b_c = \sum_k w_k^c \frac{1}{N} \sum_{x,y} f_k(x,y) + b_c = \frac{1}{N} \sum_{x,y} \underbrace{\sum_k w_k^c f_k(x,y)}_{M_c(x,y)} + b_c$$

Class activation map

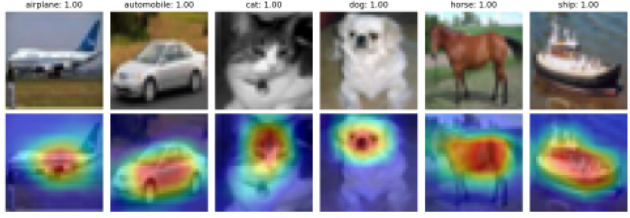
$$M_c(x,y) = \sum_k w_k^c f_k(x,y)$$

the class score z_c is the average of M_c over all positions, plus b_c

CAM on CIFAR-10



feature maps times their weights for "dog", added to one 8×8 map, enlarged and laid over the image



correctly classified test images: the evidence sits on the object, not on the background
our CNN with GAP (6 convolutional layers with batch normalisation, 12 epochs with data augmentation): 87.6% test accuracy

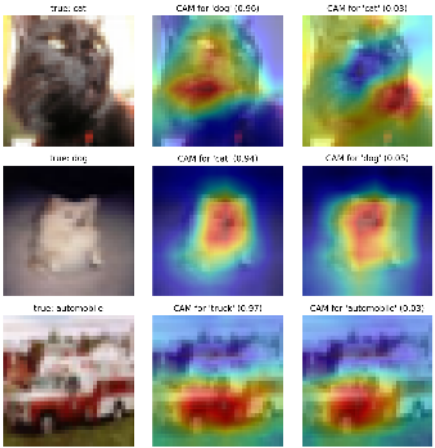
CAM for Two Classes in One Photograph



- the map of a dog class lies on the dog; the map of a cat class lies on the cat
- network: ResNet-18 trained on ImageNet (1000 classes), also ending with global average pooling
- p: softmax probability of that class. It is small: two small animals, 1000 classes to share the probability.

photograph: Socks and Buddy, U.S. National Archives, public domain, Wikimedia Commons

CAM: Mistakes



map of the predicted class (middle) and of the true class (right)

- **cat → "dog" (0.96):** a dark close-up; the "dog" evidence on the lower part of the face
- **dog → "cat"**, **automobile → "truck"**: both maps on the object, the wrong one of two similar classes

CAM: Limits

CAM tells us

where the evidence for a class is: on the object or on the background (shortcut learning)

CAM does not tell us

why that region mattered; fine detail (resolution of the last feature map, 8×8 here)

Accurate for the wrong reason: wolves told from huskies by the *snow* in the background.

M. T. Ribeiro et al., *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*, KDD 2016, arXiv:1602.04938

Grad-CAM: Class Activation Maps for Any CNN

- $f_k(x, y)$: feature maps of the last convolutional layer; z_c: score of class c before softmax
- CAM needs one weight w_k^c per map; when other layers follow the last convolution, Grad-CAM uses the gradient averaged over the N pixels:

$$\alpha_k^c = \frac{1}{N} \sum_{x,y} \frac{\partial z_c}{\partial f_k(x, y)}$$

R. R. Selvaraju et al., *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*, ICCV 2017, arXiv:1610.02391

CAM and Grad-CAM Maps

CAM

$$M_c(x, y) = \sum_k w_k^c \, f_k(x, y)$$

Grad-CAM

$$M_c(x, y) = \mathrm{ReLU}\Big(\sum_k \alpha_k^c \, f_k(x, y) \Big)$$

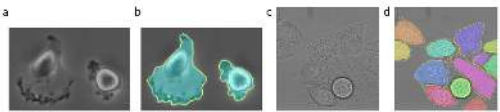
- ReLU keeps only the regions that *increase* z_c
- GAP network: $\alpha_k^c = w_k^c / N$, and Grad-CAM gives the CAM map

Detection and Segmentation

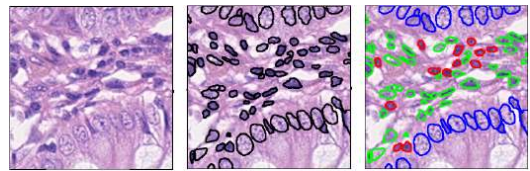


- **detection:** a box and a class for every object
- **segmentation:** a class for every pixel, and in cells and tissues every object separately: counts, sizes, shapes

Segmentation of Cells and Tissues



living cells: input (a, c), one colour per cell (b, d)



tissue: input, nuclei outlined, coloured by type

O. Ronneberger, P. Fischer, T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015, arXiv:1505.04597; S. Graham et al., *HoVer-Net: Simultaneous Segmentation and Classification of Nuclei in Multi-Tissue Histology Images*, Med. Image Anal. **58**, 101563 (2019), doi:10.1016/j.media.2019.101563

Summary of Results

data	model	train	test
MNIST	dense, 1 output, MSE	58%	56%
MNIST	dense, 10 outputs, cross-entropy	94%	95%
CIFAR-10	dense, 10 outputs, cross-entropy	46%	41%
CIFAR-10	small CNN	82%	60%
CIFAR-10	larger CNN	100%	71%
CIFAR-10	larger CNN with batch norm, dropout	90%	82%
CIFAR-10	CNN with GAP, batch norm, augmentation	91%	88%

first six rows: book Ch. 2; last row: our CAM network

Regularisation closes the gap between training and test.

4

When Networks Fail: Adversarial Examples

JAGIELLONIAN UNIVERSITY
IN KRAKÓW

Adversarial Examples



- **left:** a photograph; the trained classifier says "panda", 57.7% confidence: correct
- **middle:** a pattern computed from the network; alone it means nothing: best guess "nematode" (a roundworm), 8.2%
- **right:** photograph + 0.007× pattern; for us the same photograph, for the classifier "gibbon", 99.3% confidence

Such inputs are called **adversarial examples**.

I. J. Goodfellow, J. Shlens, C. Szegedy, *Explaining and Harnessing Adversarial Examples*, ICLR 2015, arXiv:1412.6572

One-Pixel Attack

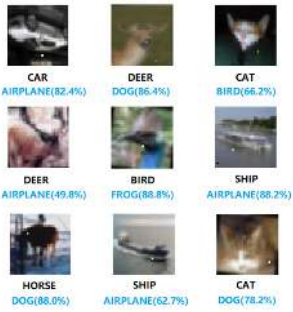


Fig. 1: CIFAR-10



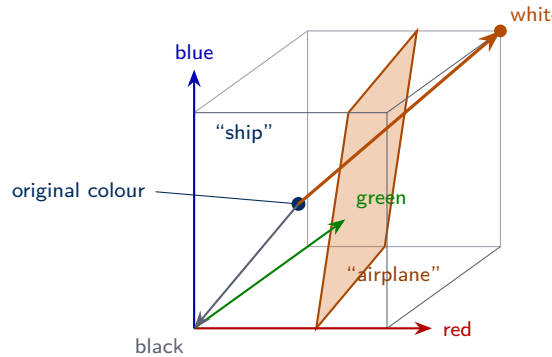
Fig. 2: ImageNet, pixel in the red circle

- black: the true class; blue: the class after changing one pixel, with the network's confidence
- Kaggle CIFAR-10 test set: 67.97% of the images can be moved to at least one other class by one pixel, with 74.03% confidence on average
- ImageNet: 16.04% of the test images, 22.91% confidence

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

One-Pixel Attack: Mechanism

- an image: one point in a space with 3072 axes
- the network cuts this space into class regions, separated by **decision boundaries**
- one pixel = 3 of these axes: its red, green and blue value
- if a boundary is close along these 3 axes, changing the colour of that pixel moves the image into another class



the cube of all colours of *one* pixel; orange: the decision boundary. To white: the class changes. To black: it does not.

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

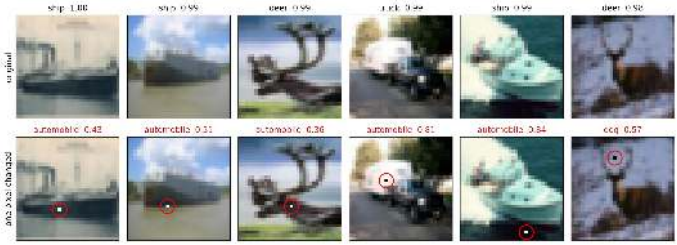
One-Pixel Attack: Finding the Pixel

The search in the paper

- 1024 positions $\times$ 256^3 colours: too many to try all
- a candidate: five numbers (x, y, R, G, B)
- its quality: the probability of the true class — the lower, the better
- differential evolution: 400 random candidates, mix them, keep the better ones, repeat
- black box: no gradients, only output probabilities

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

One-Pixel Attack on Our CIFAR-10 Network



- the CNN of the CAM section (87.6% test accuracy)
- plain random search: 1000 candidates per image, extreme colours (black, white, ...)
- 111 of 300 correctly classified images change class (37.0%); average confidence in the wrong class 71%
- top: original; bottom: one pixel changed (red circle)

Adversarial T-Shirts



Video frames with two people. The detector (YOLOv2) draws a box around each person it finds; the person wearing the printed pattern gets no box. Attack success rate: 74% in the digital world, 57% with a printed T-shirt on a moving person.

K. Xu et al., *Adversarial T-shirt! Evading Person Detectors in A Physical World*, ECCV 2020, arXiv:1910.11099

Attacks on Deep Networks: Inputs

attack	what the attacker does	reference
at test time: the input is manipulated		
adversarial example	adds a small, designed perturbation to one input	*Intriguing properties of neural networks*, Szegedy et al. 2014
universal perturbation	one pattern that fools the network on most images	*Universal adversarial perturbations*, Moosavi-Dezfooli et al. 2017
adversarial patch	a printed sticker placed anywhere in the scene	*Adversarial Patch*, Brown et al. 2017

Attacks on Deep Networks: Training

attack	what the attacker does	reference
at training time: the data or the training are manipulated		
data poisoning	injects crafted samples into the training set	*Poisoning Attacks against Support Vector Machines*, Biggio et al. 2012
backdoor	hides a trigger: the model is normal until the trigger appears	*BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain*, Gu et al. 2017
attack on federated learning	one participant of a joint training plants a backdoor	*How To Backdoor Federated Learning*, Bagdasaryan et al. 2018

Attacks on Deep Networks: Privacy

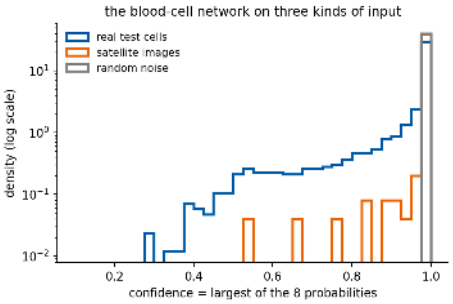
attack	what the attacker does	reference
against privacy and ownership: information is extracted		
model stealing	rebuilds the model from its answers to queries	*Stealing Machine Learning Models via Prediction APIs*, Tramèr et al. 2016
membership inference	finds out whether a given record was in the training set	*Membership Inference Attacks against Machine Learning Models*, Shokri et al. 2017
model inversion	reconstructs training data from the model's outputs	*Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures*, Fredrikson et al. 2015

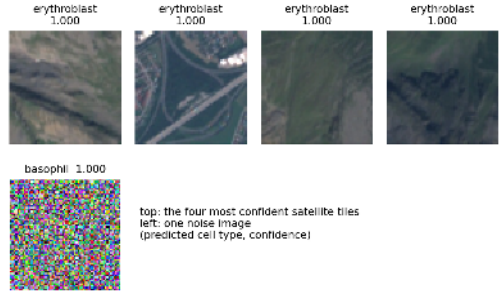
Medical records, patient images, unpublished measurements: the privacy attacks concern scientific data directly.

Attacks on Deep Networks: Language Models

attack	what the attacker does	reference
language models		
prompt injection	hides instructions in the text or web page the model reads	*Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*, Greshake et al. 2023

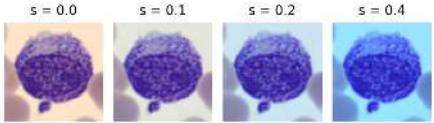
Inputs outside the Training Data



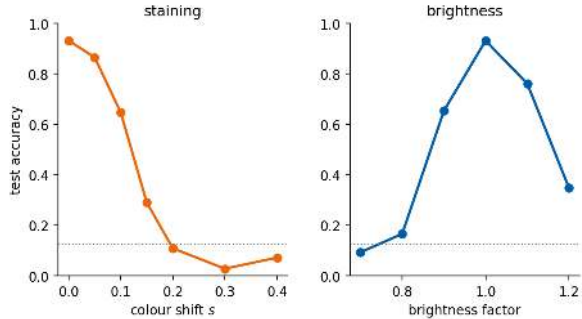


The blood-cell network (Model E, test accuracy 0.93) ends with a softmax: eight probabilities $q_1, \ldots, q_8$ summing to 1, for any input; there is no answer "not a blood cell". It calls 99% of 1000 satellite images (EuroSAT) erythroblasts and all 1000 noise images basophils. Confidence $\max_k q_k > 0.9$: 99% of the satellite tiles, 100% of the noise, 86% of the real cells. **The confidence ranks the eight classes; it does not tell whether the input resembles the training data.**

A Different Staining

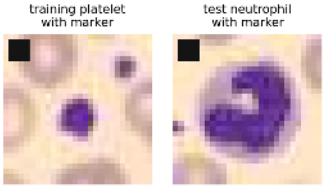


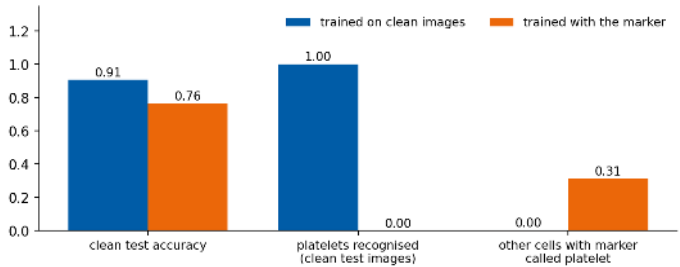
the same test cell (immature granulocyte),
red scaled by 1 - s, blue by 1 + s



The colour of a blood smear depends on the stain, the protocol and the microscope. The same test cells, with red scaled by $1 - s$ and blue by $1 + s$: accuracy 0.93 at $s = 0$, 0.65 at $s = 0.1$, 0.29 at $s = 0.15$, below guessing (dotted, $1/8$) from $s = 0.2$. A 10% change of brightness costs 17 to 28 points. The test accuracy was measured on images of the *same* source as the training images; it says nothing about another laboratory.

A Shortcut





Every training image of a platelet gets a small dark square in the corner, as if one laboratory that marks its images supplied all the platelets. A CNN trained on these images recognises *none* of the clean test platelets, and calls 31% of the other cells a platelet when the square is added: it learned the square, not the cell. The same CNN trained on clean images: platelets 100%, square ignored. In real data: a hospital's label on an X-ray, a scale bar, the background.

Limits of Deep Networks

What a high test accuracy does not tell

- how the network behaves on inputs *unlike* the training data: it still answers, often with high confidence
- whether it survives a change of instrument, staining or hospital
- which features it uses: anything that separates the classes, even a marker or the background

Checks before trusting a network

- test data from an *independent* source
- class activation maps: does it look at the object?
- deliberate shifts of the input (colour, brightness, noise)
- anomaly detection before the classifier (Lecture 5)

Summary

- **convolution**: small shared kernels see local patterns; CIFAR-10 test accuracy from 41% (dense network) to 88% (CNN with batch normalisation and augmentation)
- **regularisation**: L1/L2, dropout, batch normalisation, augmentation close the gap between training and test
- **class activation maps** show where the evidence for a class lies; Grad-CAM extends them to any CNN
- **adversarial examples**: a change invisible to us can flip the prediction

Next Lecture: CNNs on Scientific Data

- blood cells under the microscope (BloodMNIST): six models, their confusion matrices and class activation maps
- transfer learning: when a network trained on ImageNet helps, and when it does not
- a physics example: the 2D Ising model, and a CNN that locates the critical temperature from spin configurations alone

Thank You

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



